System Design Masterclass Vol. 5: High-Level Design (HLD) — Google-Scale Distributed Systems, Storage Internals, and Consensus (The Google Special)

Abstract

The definitive advanced engineering masterclass dedicated to cracking Google L4, L5 (Senior), and Staff-level Distributed Systems interviews. While standard industry system design interviews frequently evaluate conventional CRUD web tiers, Google's technical bar uniquely demands an intimate, first-principles understanding of physical hardware constraints, internal storage engines, distributed consensus protocols, and iconic Google infrastructure platforms. This volume deconstructs the foundational systems that power Google's global footprint: (1) Storage Engine Internals: Log-Structured Merge-trees (LSM-Trees) vs. B+ Trees, Write-Ahead Logs (WAL), MemTables, Immutable SSTables, sparse indexing, Bloom filters, and compaction strategies (Google BigTable, LevelDB, RocksDB); (2) Distributed Consensus & Spanner Architecture: The mechanics of Paxos and Raft, quorum consensus ($R+W > N$), distributed lock services (Google Chubby), and Google Spanner's TrueTime API with GPS/atomic clocks and the strict serializability commit-wait rule; (3) Real-Time Collaborative Editing (Google Docs): Operational Transformation (OT) vs. Conflict-Free Replicated Data Types (CRDTs), character-offset transformation state machines, and disconnection convergence; (4) Cloud Storage & Global Deduplication (Google Drive): Fixed-size vs. Content-Defined Chunking (CDC/Rabin Fingerprints), SHA-256 Content-Addressable Storage (CAS), global deduplication registries, and delta sync; (5) Distributed Web Crawler & Inverted Index (Google Search): Crawl frontier prioritization and politeness queues, URL deduplication via distributed Bloom filters, inverted index postings lists, and MapReduce batch indexing pipelines; and (6) The Google Interview Execution Rubric: Essential latency numbers every engineer must know, hardware bottlenecks, and fatal failure modes.

Index Terms

distributed systems, Google system design, LSM-tree, B+ tree, Spanner, TrueTime, Paxos, Raft, Chubby, Google Docs, Operational Transformation, CRDT, Google Drive, chunking, deduplication, web crawler, inverted index, MapReduce

1 The Google Interview Bar: Why Standard Web Architecture Fails

A standard system design interview at mid-tier companies often revolves around connecting an API Gateway to a load balancer, a microservice, a Redis cache, and a PostgreSQL database. In a Google L4/L5 interview, this approach is insufficient.

Google interviewers evaluate candidates as systems engineers who must reason about systems running across tens of thousands of commodity machines in multiple worldwide data centers. Google candidates are strictly evaluated on:

1) Hardware and Physical Constraints: Understanding the physical latency differences between CPU caches, RAM, SSDs, spinning disks, and cross-datacenter fiber optic transit.
2) Storage Engine Mechanics: Knowing exactly how bits are written to and retrieved from disk, why traditional relational engines choke on high write throughput, and how sequential append architectures function.
3) Distributed Coordination & Consensus: How machines agree on state in the presence of network partitions, packet drops, clock drift, and machine crashes without deadlocking.
4) Algorithmic Primitives at Scale: Applying specialized data structures (Bloom filters, SkipLists, Inverted Indexes, SimHash, Rolling Checksums) to process petabytes with sub-second latency.

1.1 Latency Numbers Every Google Engineer Must Know

Every quantitative trade-off in a Google interview must be anchored in physical hardware realities. Table 1 outlines the classic order-of-magnitude latencies established by Jeff Dean.

Key Architectural Takeaway: Random disk seeks are roughly $100,000\times$ slower than RAM access and $1,000\times$ slower than SSD random reads. Therefore, any system handling Google-scale writes must convert random disk writes into sequential disk appends.

2 Storage Engine Internals: LSM-Trees vs. B+ Trees

One of the most frequent deep dives in Google interviews is: "How does Google BigTable, RocksDB, or Cassandra handle 10 million writes per second while maintaining sorted access?"

TABLE 1
Physical Hardware Latency Comparison (Jeff Dean's Numbers)

Hardware Operation	Real-World Latency	Scaled to Human Time (1 ns = 1 sec)
L1 CPU Cache Reference	0.5 ns	0.5 seconds
Branch Mispredict Penalty	5 ns	5 seconds
L2 CPU Cache Reference	7 ns	7 seconds
Mutex Lock / Unlock	25 ns	25 seconds
Main Memory (RAM) Access	100 ns	1.7 minutes
Compress 1 KB with ZStandard	2,000 ns (2 μs)	33 minutes
Read 1 MB sequentially from Memory (RAM)	250,000 ns (250 μs)	2.9 days
SSD Random Read (NVMe Flash)	10,000 ns (10 μs)	2.8 hours
Read 1 MB sequentially from SSD	1,000,000 ns (1 ms)	11.6 days
Rotational Hard Disk Seek	10,000,000 ns (10 ms)	4 months
Read 1 MB sequentially from Hard Disk	20,000,000 ns (20 ms)	8 months
Round-trip within same Data Center	500,000 ns (0.5 ms)	5.8 days
Cross-Continental Round-trip (NY to London)	150,000,000 ns (150 ms)	5 years

2.1 The Flaw of Traditional B+ Trees at Scale

Relational databases (MySQL InnoDB, PostgreSQL) organize data on disk using B+ Trees:

- Data is organized into fixed-size pages (typically 4 KB or 16 KB).
- When a row is inserted, updated, or deleted, the database must locate the specific leaf page on disk and overwrite it in-place.
- The Bottleneck: In-place page updates require random disk I/O. At high write concurrency (100,000+ writes/sec), the disk arm on HDDs throngs with seeks, and even modern NVMe SSDs suffer from severe Write Amplification (overwriting an entire 16 KB page to modify a 50-byte record) and garbage collection stalls.

2.2 The Log-Structured Merge-Tree (LSM-Tree) Architecture

LSM-Trees (pioneered by Google BigTable and used in LevelDB, RocksDB, Cassandra, and ScyllaDB) solve this by trading read latency and background I/O for near-optimal sequential write throughput. An LSM-Tree consists of five primary components:

1) Write-Ahead Log (WAL): Every incoming write (insert, update, delete) is immediately appended sequentially to an append-only log on non-volatile storage. Because sequential disk writes incur minimal seek overhead, the WAL achieves maximum hardware write throughput while guaranteeing crash durability.
2) In-Memory MemTable: Concurrently with the WAL append, the key-value pair is inserted into an in-memory sorted data structure (typically a SkipList or Red-Black Tree). Because it resides in RAM, inserts and sorted range queries operate in $O(\log N)$ with nanosecond latency.
3) Immutable MemTable & Flush: When the active MemTable reaches a size threshold (e.g., 64 MB or 128 MB), it is marked immutable, a fresh MemTable is allocated for new writes, and a background thread flushes the immutable MemTable sequentially to disk as an SSTable (Sorted String Table).
4) SSTable (Sorted String Table): An immutable file on disk containing key-value pairs sorted in lexicographical order. Each SSTable contains:
    - Data Blocks: Contiguous chunks of sorted key-value pairs (e.g., 4 KB blocks).
    - Sparse Index: In-memory index storing the offset of the first key of every block. A search needs only binary-search the sparse index, then seek directly to the matching 4 KB block.
    - Bloom Filter: A probabilistic, space-efficient bit array embedded in the SSTable header that can definitively determine if a key is not present in the SSTable, avoiding unnecessary disk reads.
5) Compaction (Merge-Sort on Disk): Because SSTables are immutable, updates do not modify existing files, and deletes simply write a special marker called a Tombstone. Over time, multiple SSTables accumulate duplicate keys and tombstones. A background compaction thread continuously performs a multi-way merge-sort across SSTables, purging tombstones and duplicate versions, outputting newly consolidated, sorted SSTables.

2.3 The LSM Read Path: Resolving the Multi-File Search

When a client requests GET key:

1) Search the active in-memory MemTable. If found, return immediately ($< 1\,\mu$s).
2) Search any flushing Immutable MemTables in RAM.
3) If not found in RAM, inspect the on-disk SSTables from newest to oldest:

TABLE 2
Comprehensive Comparison: B+ Tree vs. LSM-Tree Storage Engines

Metric / Dimension	B+ Tree (e.g., MySQL, Postgres)	LSM-Tree (e.g., BigTable, RocksDB, Cassandra)
Primary Workload Optimization	Read-heavy workloads (> 80% reads)	Write-heavy workloads (> 50% writes, high ingestion)
Write Path Mechanism	In-place page update (Random I/O)	WAL append + MemTable insert (Pure Sequential I/O)
Write Throughput	Limited by random disk seek / page writes	Maximizes raw sequential hardware bandwidth
Write Amplification	High (overwrites full 16 KB page for small row)	Moderate to Low (amortized across batch flushes)
Read Latency (Point Query)	Fast: exactly 1 or 2 page lookups ($O(\log N)$)	Slower: may search MemTable + multiple SSTables
Read Mitigation	Buffer pool caching	Bloom Filters + Block Cache + Leveled Compaction
Concurrency Control	Complex latches/locks on tree nodes	Lock-free SkipList writes; immutable disk files
Space Efficiency	Page fragmentation (50% to 70% fill factor)	High compression ratio (sorted data compresses well)

- Query the SSTable's Bloom Filter. If the Bloom filter returns false, the key is guaranteed not in this file; skip disk read entirely!
- If true, binary search the in-memory Sparse Index to locate the block offset.
- Read the specific 4 KB block from disk and binary search within the block.

4) Return the newest valid value (or null if the newest entry is a tombstone).

3 Distributed Consensus, Leader Election, & Google Spanner

At Google scale, single-leader databases fail when the leader crashes or the network partitions. Standard replication (Master-Replica) suffers from split-brain scenarios or data loss during failover.

3.1 The Consensus Problem & Quorum Arithmetic

Consensus algorithms ensure that a distributed group of machines agrees on a sequence of state machine operations, even when up to F nodes crash or experience network partitions in a cluster of $N = 2F + 1$ machines.

Quorum Arithmetic: In an N-node cluster:

- Write Quorum (W): Number of nodes that must acknowledge a write before success.
- Read Quorum (R): Number of nodes that must be queried during a read.
- Strict Consistency Rule: If $W + R > N$, the read quorum and write quorum are mathematically guaranteed to overlap on at least one node containing the latest update.
- Example: In a 5-node cluster ($N = 5$), setting $W = 3$ and $R = 3$ guarantees strong consistency ($3 + 3 = 6 > 5$).

3.2 Paxos and Raft: The Two Production Standards

- Raft: Decomposes consensus into two explicit phases: (1) Leader Election using randomized election timeouts (150–300 ms) and heartbeats; and (2) Log Replication where the leader appends entries to follower logs. An entry is committed once replicated across a majority quorum ($N/2 + 1$).
- Paxos: The theoretical foundation used by Google. Classic Paxos operates in two rounds: Phase 1 (Prepare/Promise) to establish leadership proposal numbers, and Phase 2 (Accept/Accepted) to commit the value. Multi-Paxos optimizes this by electing a stable leader, amortizing Phase 1 across thousands of writes.

3.3 Google Chubby: Distributed Locks and Coordination

Google Chubby is a coarse-grained distributed lock service based on Paxos. While ZooKeeper and etcd are open-source equivalents, Google interviewers frequently ask how Chubby coordinates systems:

- A Chubby cell consists of 5 replicas running Paxos. One replica is elected the master with a timed lease.
- Use Cases: Master election for Google BigTable and GFS, location of root metadata tablets, and distributed mutual exclusion via advisory locks with sequence numbers (fencing tokens) to prevent split-brain zombies from corrupting storage.

3.4 Google Spanner & The TrueTime API

Google Spanner is Google's globally distributed NewSQL database that provides ACID transactions, external consistency (linearizability), and SQL querying across continents.

3.4.1 The Fundamental Clock Problem

Standard servers synchronize time via Network Time Protocol (NTP). However, NTP over public networks or cross-datacenter WAN experiences variable latency, causing server clocks to drift by 100 ms to 250 ms. If Node A assigns timestamp $t_1 = 100$ and Node B assigns $t_2 = 90$ for a later transaction, linearizability is violated!

3.4.2 The TrueTime Solution

Google solved this by integrating physical hardware directly into every Google data center:

- Every datacenter cluster contains a set of Time Master machines equipped with two independent time references: (1) GPS receivers with dedicated antennas, and (2) Atomic Clocks (rubidium oscillators).
- GPS and Atomic clocks fail independently: GPS suffers from antenna failures or weather; atomic clocks drift slowly over time. Pairing them eliminates common failure modes.
- The TrueTime API: Instead of returning a single scalar time t, TrueTime returns an uncertainty interval:

$$\text{TT.now}() = [t_{\text{earliest}}, t_{\text{latest}}] \quad \text{where} \quad \epsilon = \frac{t_{\text{latest}} - t_{\text{earliest}}}{2} \tag{1}$$

  The parameter ϵ represents the guaranteed maximum clock drift bound, typically kept under 1 ms to 7 ms.

3.4.3 The Commit-Wait Rule: Guaranteeing Strict Serializability

To guarantee that a transaction T_2 that begins after transaction T_1 commits will receive a strictly larger timestamp ($s_2 > s_1$), Spanner enforces the Commit-Wait Rule:

1) When transaction T_1 prepares to commit, the coordinator requests TT.now() and chooses commit timestamp $s_1 = t_{\text{latest}}$.
2) The Wait Period: The coordinator deliberately delays returning success to the client until it is physically certain that the absolute real-world time has passed s_1:

$$\text{Wait until TT.now().earliest} > s_1 \tag{2}$$

3) Because the coordinator waits for at least 2ϵ (e.g., 8 ms to 14 ms), any subsequent transaction T_2 started anywhere on Earth is guaranteed to pick an absolute timestamp $s_2 > s_1$.
4) Interview Value: Spanner achieves global, lock-free, strongly consistent snapshot reads across continents without requiring distributed locking across data centers!

4 Design Google Docs: Real-Time Collaborative Editing

4.1 Requirements & Operational Challenges

- Functional Requirements: Multiple users can concurrently type, delete, and format text within the same document in real time. Cursor locations of other users are visible. Support offline editing with automatic reconciliation upon reconnecting.
- Non-Functional Requirements: Sub-100 ms local character rendering (zero perceived typing latency), eventual convergence (all clients must arrive at the identical character sequence), high availability, and horizontal scalability.
- The Challenge: Two users type at position 3 at the exact same physical millisecond. Without synchronization, Client A sees “ABC”, while Client B sees “ACB”.

4.2 Operational Transformation (OT) vs. CRDTs

There are two primary paradigms for real-time collaboration:

4.2.1 Operational Transformation (OT) — Google Docs’ Architecture

OT relies on a centralized server that acts as the single ordering authority. Operations are represented as triplets: Insert(pos, char) or Delete(pos). When two concurrent operations A and B are executed, the server applies a transformation function $T(A, B) \to (A', B')$ such that:

$$\text{Doc} \circ A \circ B' \equiv \text{Doc} \circ B \circ A' \tag{3}$$

Concrete Walkthrough:

- Initial Document: ”CAT” (indices 0, 1, 2).
- User 1 (Client 1) inserts ’H’ at index 1 $\to$ wants ”CHAT”. Operation $O_1 = \text{Insert}(1, \text{'H'})$.
- User 2 (Client 2) inserts ’S’ at index 3 $\to$ wants ”CATS”. Operation $O_2 = \text{Insert}(3, \text{'S'})$.
- Both operations are generated concurrently against base revision 0.
- The server receives O_1 first: Document becomes ”CHAT” (Revision 1). The server broadcasts O_1 to Client 2.
- When O_2 arrives at the server, the server detects that O_2 was generated against Revision 0, but the document is now at Revision 1.
- The server transforms O_2 against O_1: because an extra character ’H’ was inserted before index 3, the target index for O_2 must be shifted right by +1:

$$O_2' = T(O_2, O_1) = \text{Insert}(3 + 1, \text{'S'}) = \text{Insert}(4, \text{'S'}) \tag{4}$$

- The server applies O_2', document becomes ”CHATS” (Revision 2), and broadcasts O_2' to Client 1. Both clients converge identically!

4.2.2 Conflict-Free Replicated Data Types (CRDTs)

CRDTs (used by Figma and Apple Notes) eliminate the central transformation server by giving every character a globally unique, fractional position identifier (e.g., between position 1.0 and 2.0, generate position 1.5). Operations are mathematically commutative and associative ($A \cup B = B \cup A$). Trade-off: CRDTs eliminate the centralized ordering server, but each character carries a large metadata overhead (UUID, Lamport timestamp, fractional vector), consuming 5× to 10× more memory than plain strings.

4.3 Google Docs End-to-End System Architecture

1) Client Layer:
   - Renders keystrokes locally with 0 ms latency in an optimistic local buffer.
   - Maintains three operational states: Synchronized, Awaiting ACK, and Awaiting ACK with Pending Local Operations.
2) WebSocket Gateway / Connection Manager: Terminates persistent bidirectional WebSockets for active document sessions.
3) Document Session Manager (In-Memory Worker):
   - Holds the canonical document in RAM.
   - Assigns monotonically increasing revision sequence numbers ($R_1, R_2, \dots$).
   - Runs the OT transformation engine and broadcasts transformed operations to all connected session participants.
4) Persistence Layer:
   - Operations Log (Google Spanner / BigTable): Append-only log of every transformed operation: (doc_id, rev_id, user_id, op
   - Periodic Snapshot Storage (Colossus / GCS): Every 500 operations, the document state is compressed and saved as a snapshot. Loading a document fetches the latest snapshot and replays only subsequent delta operations.

5 Design Google Drive: Chunking, Deduplication, and Delta Sync

5.1 Scale and Requirements

- Scale: 1 billion users, 100 million daily active users, petabytes of daily file uploads.
- Core Capabilities: Upload/download files up to 50 GB; instant cross-user deduplication; delta sync (modifying 1 paragraph of a 1 GB document uploads only the changed bytes); seamless resume after network interruption.

5.2 Content-Addressable Storage (CAS) & Chunking

Storing 50 GB files as monolithic objects causes massive network re-transmissions upon connection drops and prevents deduplication. Google Drive chunks files:

1) Fixed-Size Chunking (e.g., 4 MB): The file is sliced every 4 MB. Simple, but inserting 1 byte at the start of a file shifts all subsequent chunk boundaries, invalidating all downstream chunk hashes.
2) Content-Defined Chunking (CDC via Rabin Fingerprints / FastCDC): A rolling hash slides across the byte stream. A chunk boundary is declared whenever the lowest N bits of the rolling hash match a pattern (e.g., hash & $0x1FFF == 0$). If data is inserted in the middle, only the local chunk boundary shifts; all subsequent chunk boundaries remain identical!
3) Content-Addressable Storage (CAS): Each chunk is hashed using SHA-256. The chunk's name in Google Colossus/GCS is its SHA-256 hash:

$$\text{ChunkID} = \text{SHA-256}(\text{ChunkBytes}) \tag{5}$$

   Because identical content produces identical hashes, duplicate chunks across different files (or different users) share the exact same physical block!

5.3 Global Deduplication Workflow

When a user uploads a 5 GB ISO image:

1) The client computes the file's overall hash and the SHA-256 hashes of all its 4 MB chunks.
2) Client calls: POST /files/verify-chunks with the list of chunk hashes.
3) The Metadata Service queries the Global Chunk Registry:
   - If User A already uploaded this file, 100% of the chunk hashes already exist in Colossus.
   - The server responds: "All chunks exist."
   - The upload finishes in 0 milliseconds! The server merely creates a new metadata entry linking the new user's namespace to the existing chunk IDs.
4) If only 2 out of 1,200 chunks are missing, the server returns pre-signed upload URLs for only those 2 missing chunks, saving 99.8% of network bandwidth!

5.4 End-to-End System Components

- Client Sync Agent: Local desktop daemon with:
  - File Watcher: Monitors OS filesystem events (e.g., inotify on Linux, FSEvents on macOS).
  - Chunker & Hasher: Slices files using FastCDC and computes SHA-256 hashes.
  - Local SQLite DB: Caches file paths, local modification timestamps, chunk hashes, and sync state.
- Metadata Service (Spanner / CockroachDB): Stores the hierarchical file system structure, permissions, file versions, and the ordered list of chunk hashes per file.
- Block Storage Service (Google Colossus / S3): Highly available object store holding raw, encrypted, immutable data chunks.
- Notification Service (WebSockets / Push): Emits change notifications to all connected devices belonging to the user when a file is modified.

6 Design Google Search: Web Crawler & Inverted Index

6.1 System Scope & Quantitative Scale

- Ingest 10 billion web pages; query volume: 100,000 queries per second (QPS); median search latency: < 200 ms.

6.2 The Distributed Web Crawler Architecture

1) URL Frontier (Prioritization and Politeness):
   - Priority Queues: Order URLs by importance (PageRank, domain authority, update frequency).
   - Politeness Queues: Ensure crawler does not overwhelm target web servers. Each target domain maps to a dedicated queue with a rate limiter (e.g., at most 1 request every 1,000 ms, obeying robots.txt).
2) DNS Resolver Cache: Standard DNS resolution takes 20–100 ms. A high-throughput local DNS cache is required to resolve millions of hostnames in RAM without bottlenecking network threads.
3) URL Seen Service (Distributed Bloom Filter): To avoid crawling the same URL twice across 10 billion pages:
   - Storing 10B URLs in memory (100 bytes $\times$ 10^{10} = 1 TB) is expensive.
   - A Bloom Filter with 1% false positive rate requires only 9.6 bits per item $\approx$ 12 GB of RAM! If the Bloom filter returns false, the URL is definitely new. If true, a fallback disk check in BigTable verifies true existence.
4) Content Deduplication via SimHash: Web pages with identical text but different banner ads are flagged as near-duplicates using SimHash (64-bit fingerprints where Hamming distance ≤ 3 indicates duplicate content).

6.3 The Inverted Search Index: Building & Serving

Search queries cannot scan raw documents. The system builds an Inverted Index:

$$\text{Forward Index:} \quad \text{DocID} \longrightarrow [\text{word}_1, \text{word}_2, \text{word}_3, \dots] \tag{6}$$

$$\text{Inverted Index:} \quad \text{Word} \longrightarrow \text{Postings List: } [(\text{Doc}_1, \text{tf}, [\text{pos}_1]), (\text{Doc}_4, \text{tf}, [\text{pos}_2]), \dots] \tag{7}$$

6.3.1 Batch Indexing via MapReduce

- Map Phase: Reads raw web documents and emits key-value pairs:

$$\text{Input: (DocID, Text)} \quad \longrightarrow \quad \text{Emit: (Term, (DocID, Position))} \tag{8}$$

- Shuffle & Sort: Partitions by Term and sorts all document hits alphabetically.
- Reduce Phase: Aggregates all document hits for a term into a compressed Postings List and calculates Term Frequency (TF) and Inverse Document Frequency (IDF):

$$\text{TF-IDF}(\text{term}, \text{doc}) = \text{TF}(\text{term}, \text{doc}) \times \log\left(\frac{N}{\text{DF}(\text{term})}\right) \tag{9}$$

6.3.2 Query Serving Pipeline (Scatter-Gather Architecture)

When a user searches for "distributed systems":

1) Query Parser: Tokenizes query, applies stemming (e.g., "distributing" $\rightarrow$ "distribute"), and removes stop-words.
2) Index Aggregator (Scatter): Broadcasts the query terms to K distributed Index Shard Servers.
3) Leaf Index Nodes: Intersect the postings lists for "distributed" and "systems". They retrieve candidate Document IDs and compute preliminary BM25 / TF-IDF relevance scores.
4) Aggregator (Gather): Combines the top 1,000 candidates from all shards.
5) Ranking Service: Evaluates candidates using high-dimensional ML ranking models combining BM25, PageRank, user location, and freshness.
6) Snippet Generator: Pulls document text summaries from BigTable and returns the top 10 results in < 150 ms.

7 The Google Interview Rubric & Anti-Pattern Checklist

7.1 Google Hiring Committee Evaluation Rubric

Google evaluates candidates across four calibrated dimensions:

1) Analytical Problem Solving (25%): Clarifying ambiguous requirements, deriving constraints, and executing accurate back-of-the-envelope capacity estimations.
2) System Architecture & Trade-Offs (35%): Justifying every design decision with explicit trade-offs (e.g., choosing LSM over B+ Tree, Paxos over 2PC, OT over CRDT).
3) Reliability, Failure Modes, & Scale (25%): Proactively identifying single points of failure (SPOFs), network partitions, data corruption, thundering herds, and explaining exact recovery mechanisms.
4) Infrastructure & Hardware Literacy (15%): Demonstrating first-principles knowledge of memory vs. disk seek latency, network bandwidth limits, and storage mechanics.

7.2 Fatal Anti-Patterns That Eliminate Candidates at Google

- Hand-Waving Database Magic: Stating "the database will handle replication and consistency" without knowing if the underlying engine uses Raft, Paxos, master-replica, or multi-master.
- Ignoring Disk Seek Bottlenecks: Proposing relational databases for write workloads of 1,000,000 writes/sec without acknowledging random disk seek bottlenecks and write amplification.
- Assuming Clocks Are Synchronized: Relying on server system timestamps for ordering events across data centers without accounting for NTP clock drift.
- Ignoring Failure Domains: Failing to design for datacenter outages, network partitions (split-brain), or cascading crashes under heavy load.